

Atividade Extra 1 - PLN

Nome: Mauro do Prado Santos

Questão 1

No **Google Colab** são executadas as etapas de instalação/importação das bibliotecas, montagem do Drive, leitura do PDF, normalização do texto, divisão em chunks, criação do `chunk_store`, cálculo do fingerprint, criação e consulta do índice FAISS, além da leitura e gravação dos arquivos de cache no Drive.

Fora do Colab ficam os serviços externos: o **Google Drive**, usado como armazenamento persistente, e a **API da OpenAI**, usada para gerar embeddings e respostas com modelo generativo.

Essa separação é necessária porque o Colab executa o código, mas não mantém dados de forma permanente após a sessão. O Drive resolve a persistência, e a OpenAI fornece modelos que não rodam localmente. A vantagem prática é combinar processamento local barato e controlável com serviços externos especializados, evitando recomputar o índice sempre que o notebook for reiniciado.

Questão 2

Quem gera os arquivos `faiss.index`, `config.json`, `chunk_store.pkl` e `chunk_ids.pkl` é o próprio código do notebook, mais especificamente a função `save_cache`.

Eles são criados na etapa de **rebuild** do pipeline, quando não existe cache válido ou quando o PDF/configuração mudou. O fluxo é:

PDF → chunks → embeddings via OpenAI → índice FAISS → salvamento no Drive

Função geral dos arquivos:

- `faiss.index`: armazena o índice vetorial usado na busca semântica.
 - `chunk_store.pkl`: guarda o texto dos chunks e seus metadados.
 - `chunk_ids.pkl`: mantém o mapeamento entre posições do FAISS e IDs reais dos chunks.
 - `config.json`: guarda fingerprint, data de salvamento e informações do índice.
-

Questão 3

O FAISS é responsável pela **busca vetorial eficiente**.

Ele armazena os vetores de embeddings dos chunks do PDF. Durante uma consulta, a pergunta do usuário também é transformada em embedding. Esse vetor da pergunta é comparado com os vetores armazenados no FAISS, que retorna os chunks mais semelhantes semanticamente.

Sua importância é a eficiência: em vez de comparar a pergunta manualmente com todos os textos do PDF, o FAISS faz uma busca rápida pelos trechos mais relevantes.

Questão 4

Na primeira execução com um PDF novo, o sistema executa este fluxo:

1. Monta o Google Drive.
 2. Localiza o PDF pelo caminho definido.
 3. Lê o PDF página por página com `PyPDFLoader`.
 4. Normaliza o texto extraído.
 5. Divide o conteúdo em chunks por tokens.
 6. Atribui `chunk_id` a cada fragmento.
 7. Cria o `chunk_store`, a lista `chunk_ids` e a lista de textos.
 8. Calcula o fingerprint do PDF e da configuração.
 9. Verifica se já existe cache válido.
 10. Como é um PDF novo, gera embeddings dos chunks usando `text-embedding-3-small`.
 11. Normaliza os vetores.
 12. Cria o índice FAISS com `IndexFlatIP`.
 13. Salva `faiss.index`, `chunk_store.pkl`, `chunk_ids.pkl` e `config.json` no Google Drive.
-

Questão 5

Quando o usuário envia uma pergunta:

1. A pergunta é recebida no loop interativo.
2. O sistema gera um embedding da pergunta usando a API da OpenAI.

3. Esse vetor é normalizado.
4. O FAISS busca os chunks mais semelhantes à pergunta.
5. O sistema filtra os resultados pelo `min_score`.
6. Os trechos recuperados são formatados como contexto, com indicação de fonte e página.
7. A pergunta e os trechos relevantes são enviados ao modelo generativo `gpt-4.1-mini`.
8. O modelo gera a resposta usando apenas as fontes fornecidas.
9. A resposta final é exibida ao usuário com citações no formato `[Fonte X]`.

As etapas de busca no FAISS, montagem do contexto e controle do fluxo ocorrem no Colab. As chamadas para embeddings e geração textual ocorrem na OpenAI.

Questão 6

O modelo de embeddings usado é:

`text-embedding-3-small`

Ele é usado em dois momentos:

- Para transformar os chunks do PDF em vetores semânticos.
- Para transformar a pergunta do usuário em vetor durante a consulta.

Esse tipo de modelo é adequado porque representa textos por significado, permitindo recuperar trechos relevantes mesmo quando a pergunta não usa exatamente as mesmas palavras do PDF.

Questão 7

O modelo de embeddings e o modelo generativo têm funções diferentes.

O modelo de embeddings, `text-embedding-3-small`, transforma textos em vetores numéricos. Ele não escreve respostas; serve para medir similaridade semântica entre pergunta e trechos do PDF.

O modelo generativo, `gpt-4.1-mini`, recebe a pergunta e os trechos recuperados, interpreta o contexto e produz a resposta final em linguagem natural.

Ambos são necessários: o embedding encontra as evidências relevantes; o modelo generativo transforma essas evidências em uma resposta compreensível.

Questão 8

Os principais ganhos do RAG são:

- respostas baseadas no conteúdo real do PDF;
- redução de alucinações;
- possibilidade de citar fontes;
- menor custo, pois só trechos relevantes são enviados ao modelo;
- atualização simples: basta trocar ou reindexar o documento.

Sem o mecanismo de recuperação, o modelo generativo teria que responder apenas com seu conhecimento interno ou receber o PDF inteiro como contexto. Isso reduziria a precisão, aumentaria custo e dificultaria a rastreabilidade das respostas.

Questão 9

PDF estimado:

- 10 páginas × 450 palavras = **4.500 palavras**
- Aproximação: 1 palavra ≈ 1,33 token
- Total ≈ **6.000 tokens**

Custo do `text-embedding-3-small`: $6.000 / 1.000.000 \times 0,02 = 0,00012$ Custo aproximado: **US\$ 0,00012** para gerar embeddings do PDF.

Com sobreposição entre chunks, pode ficar um pouco maior, algo em torno de **US\$ 0,00014 a US\$ 0,00015**.

Questão 10

Hipóteses:

- O PDF inteiro tem cerca de **6.000 tokens**.
- O sistema usa RAG, então não envia o PDF inteiro.
- O notebook recupera até `top_k = 8` chunks.
- Cada chunk tem cerca de **220 tokens**.
- Contexto recuperado: $8 \times 220 \approx 1.760$ tokens.
- Com pergunta, instruções e formatação: aproximadamente **2.000 tokens de entrada**.

- Resposta estimada: **300 tokens de saída.**

Custo de entrada: $2.000 / 1.000.000 \times 0,80 = 0,0016$ Custo de saída: $300 / 1.000.000 \times 3,20 = 0,00096$ Custo total estimado por pergunta: $0,0016 + 0,00096 = 0,00256$

Resposta: aproximadamente **US\$ 0,0026 por pergunta**, ou seja, cerca de **0,26 centavo de dólar por consulta**.